House of Edtech

Fullstack Developer — Fulltime Assignment 2

Version 2.1 — April 2026 (current assignment)

We are seeking highly skilled and motivated individuals proficient in modern web development. Candidates must be proficient in Node.js and capable of building efficient, scalable, and secure backend solutions, including asynchronous programming, RESTful APIs, database integration (SQL/NoSQL), middleware, and package management.

Expertise in Next.js is crucial, encompassing server-side rendering (SSR), static site generation (SSG), API routes, optimization, routing, data fetching, and deployment.

A strong understanding of basic React.js development is essential, covering **component-based architecture**, **state management** (Hooks, Context API, Query Params), lifecycle methods, and the virtual DOM.

Overall, candidates should possess a comprehensive understanding of full-stack development, from concept to deployment. This includes building robust, scalable, fast, secure, and user-friendly applications while adhering to best practices for code quality and maintainability. Experience with AI technologies is a plus.

Task

Develop a Local-First, Collaborative Document Editor with Offline Synchronization, Deterministic Conflict Resolution, and Granular Version Control. This application should function seamlessly without an internet connection, automatically reconciling state when the network resolves, and allowing users to navigate a robust document history.

What do we expect from you?

We don't expect a basic CRUD application. We want you to showcase your intellect and approach to developing impactful applications, demonstrating sophisticated problem-solving and innovative solutions beyond just technical proficiency.

Your project should reflect critical thinking, analytical capabilities, and strategic design for valuable, transformative outcomes, from ideation to scalable implementation.

Please don't build to-do lists, task managers, and basic CRUD applications. We expect you to tackle complex distributed systems problems, specifically browser-based memory management, state synchronization race conditions, and designing complex data merging algorithms over real-time communication protocols.

Mandatory Skills

Next JS 16, React.js, Git, Tailwind CSS, PostgreSQL

Good to Have Skills

Hands-on Experience with AI libraries like AI-SDK, OpenAI, Gemini or Groq!

Key Requirements

Technology Stack

- **Backend/Frontend:** Next.js 16 (leveraging TypeScript for type safety and code maintainability).

- **Database:** Select a database management system based on project needs (e.g., PostgreSQL, MySQL, MongoDB).

Functionality

- **Local-First Architecture:** Implement a robust client-side storage mechanism as the primary source of truth. The user must be able to open, edit, and close documents with zero network requests blocking the UI.
- **Background Sync Engine:** Develop a robust queueing and synchronization system. When the user regains connection, the system must push local changes to the server and fetch remote changes without overwriting or destroying the user's offline work.
- **Version History & Time Travel:** Implement a versioning system where users can capture specific snapshots of their document. They must be able to view a timeline of past versions and restore the document to a previous state safely, without corrupting the current shared document state for other active collaborators.
- **Robust Data Validation:** Ensure the server strictly validates incoming synchronization payloads to prevent malicious data from crashing the collaboration server or corrupting the document.
- **User Interface:** Design a clean, intuitive, and responsive user interface that aligns with Next.js 16's component-based architecture and styling capabilities. You can use anything related to Tailwind CSS (e.g. shadcn, radix-ui) for designing elements. Consider accessibility best practices to cater to users with diverse needs.
- **Use AI for add-on features:** Leverage AI for a wide array of add-on features. AI is a game-changer, revolutionizing various industries and aspects of daily life. Its integration allows for enhanced capabilities, providing a significant competitive edge and opening up new possibilities for innovation and efficiency.
- **Deployment:** Deploy the application to a suitable hosting platform (e.g., Vercel, Netlify). Configure continuous integration and continuous delivery (CI/CD) for streamlined development and deployment processes.
- **Code Optimization:** Optimize code for performance and efficiency, considering techniques like code splitting, caching, and server-side rendering (SSR). Write clean, well-structured, and well-documented code that adheres to best practices and style guides.
- **Real-World Considerations:** Address potential real-world challenges that might arise in a production environment, such as scalability, error handling, and security.

Must Have

- **Authentication and Authorization:** Implement a secure authentication mechanism (e.g., JWT-based, NextAuth/Auth.js). Enforce granular authorization rules to control user access. A document must support roles: Owner, Editor, and Viewer. Viewers must not be able to push state updates to the real-time server.
- **Security:** Discuss mitigation strategies and contingency plans for these challenges. How do you prevent a malicious actor from sending a massive, malformed synchronization payload that OOMs (Out of Memory) your server? Secure your API routes and implement Row Level Security (RLS) or strict ORM scoping in PostgreSQL to ensure tenant isolation.

Good to Have

- **Testing:** Consider integration and end-to-end testing for comprehensive coverage.

Evaluation Criteria

- **Functionality:** Completeness and correctness of the offline-sync process, deterministic conflict resolution merging without data loss, functional version history, data validation, authentication, and authorization.
- **User Interface:** User-friendliness, responsiveness, real-time connection status indicators, and adherence to accessibility best practices.
- **Code Quality:** Code structure, management of complex state synchronization logic, documentation, and optimization techniques (especially preventing client-side lag during rapid typing).
- **Testing:** Coverage and effectiveness of unit, integration, and end-to-end tests, specifically around the local-first sync engine.
- **Deployment:** Successful deployment and CI/CD configuration.
- **Real-World Considerations:** Demonstrated understanding of potential architectural challenges and proposed solutions (e.g., handling document state size over time).

Submission Guidelines

- Share your GitHub Repository along with the Live Deployment.
- Make sure your name, Github Profile and LinkedIn profile is mentioned in the footer of the application.